Project - 2

Title: Student Registration System with Database Integration

Project Description

Upgrade the student registration web application from Project1 to use a relational database for persistent data storage. The system will use **HTML** for structure, **CSS** for styling, **PHP** for server-side logic, and **MySQL** (via PDO) to securely store, retrieve, and manage student records.

Project Requirements

1. Database Setup & Architecture

Create a MySQL database named student_management containing a table named students.

- **Table Fields:**
    - id: Primary key, auto-incrementing integer.
    - student_name: Varchar, required.
    - email: Varchar, required, unique.
    - student_number: Varchar/Integer, required, unique.
    - year_of_study: Integer, required.
    - batch_name: Varchar, required.
    - created_at: Timestamp, defaults to current time.
- **Deliverable:** Include a database.sql file in your repository containing the queries used to create the database and table.

2. Registration Form & Backend Validation

Enhance the HTML form from project 1 to include robust server-side processing.

- **Form Structure:** Keep the same fields (Name, Email, Student Number, Year, Batch) using the POST method.
- **PHP Processing:**
    - Connect to the MySQL database securely using **PHP Data Objects (PDO)**.
    - Sanitize and validate all incoming inputs (e.g., ensure the email format is valid).
    - Use **prepared statements** to insert the data into the database to prevent SQL Injection attacks.
    - Display a success or error message on the screen after submission.

3. Displaying & Managing Registered Students

Replace the file-reading logic with live database queries.

- **Data Retrieval:** Fetch all records from the students table using an SQL SELECT query.
- **Data Presentation:** Display the retrieved student profiles cleanly in an HTML table.

- **New Feature - Delete Action:** Add a "Delete" button/link for each student row in the table. When clicked, it should remove that specific record from the database based on its unique id and refresh the list.

4. Styling & User Experience

- Maintain a professional UI layout using CSS.
- Provide clear visual feedback messages (e.g., green alerts for successful registration/deletion, red alerts for duplicate entry errors like existing student numbers).

Submission Requirements

1. GitHub Repository

Upload the updated project files to your GitHub repository. The repository must include:

- **Project Code:** All HTML, CSS, and PHP files organized cleanly.
- **Database Script:** The database.sql export file.
- **README.md:** An updated Markdown file explaining how to set up the database locally, configuration steps (like modifying database credentials in the PHP code), and project features.

2. Short Demonstration Video

Record a 3–4 minute video showing:

- **Database Connection:** A quick look at your PHP connection code and your database structure in phpMyAdmin (or your preferred database tool).
- **Application Workflow:** Registering a new student, verifying they appear instantly in the HTML table, and demonstrating the deletion of a record.
- **Code Walkthrough:** A brief explanation of your database insertion logic and how you used prepared statements.
- Submit the video via a YouTube URL or a shared Google Drive link.